Automated Verification for Health Economic Models in R: A Hands-On Tutorial

Nathan Green, *UCL*
Daniel Gallardo, *ICON Clinical Research*
Felicity Lamrock, *Queen's University Belfast*

November 9, 2026

Abstract

Health economic and decision-analytic models have become increasingly complex, contributing to a shift toward open-source programming languages like R. While best-practice guidelines consistently emphasize the conceptual importance of model verification and validation, there is a lack of practical, hands-on guidance showing *how* to implement automated software testing techniques within health economic modelling workflows. This tutorial provides a step-by-step guide to integrating automated verification tests into health economic modelling in R and the *testthat* package for more trustworthy and robust code.

Keypoints

- As health economic models increasingly transition to open-source programming languages like R, there is a critical need for practical guidance on how to systematically check complex code for hidden errors.
- This tutorial provides a step-by-step framework for health economists to adopt standard software engineering practices—specifically automated unit testing—to verify that their models are mathematically and logically accurate.
- Adopting these rigorous quality-assurance workflows will proactively prevent silent errors, improve transparency, and increase decision-maker confidence in the models used to determine healthcare reimbursement.

1 Introduction

Health economic modelling has undergone a rapid methodological paradigm shift in recent years. Decision-analytic modelling has historically been dominated by spreadsheet software such as Microsoft (MS) Excel, particularly in the UK [1]. However, as health economic models have grown in structural complexity and computational demand, driven in part by increasing microsimulation [2] and rigorous Probabilistic Sensitivity Analysis (PSA), the field has increasingly migrated toward open-source programming languages such as R. The transition offers undeniable benefits to the health economics community, including superior computational efficiency, advanced statistical capabilities, and the ability to seamlessly integrate data synthesis, model execution, and post-processing with advanced visualisations. [3]. Previous tutorials provide step-by-step guidance to help with this transition [4–6].

Health technology assessment (HTA) bodies, such as the National Institute for Health and Care Excellence (NICE), rely heavily on submitted models to inform reimbursement decisions. Given the profound financial and population health implications of these decisions, it is critical that the underlying models are technically sound [1]. Decision-makers must have confidence in both the conceptual model and its computational implementation [7]. The health economics community has long recognized the importance of model verification and validation. Prominent frameworks, including those developed by the International Society for Pharmacoeconomics and Outcomes Research and the Society for Medical Decision Making (ISPOR-SMDM) Modelling Good Research Practices Task Force [8–10], consistently emphasize the need to rigorously model check logic and internal consistency. Zorginstituut Nederland (ZIN) is currently the most explicit regulatory body globally regarding R-based submissions. They published a guideline for building cost-effectiveness models in R which mandates strict standardization to ensure that models are transparent, fully executable, and adaptable. [11] Furthermore, recent literature has provided excellent conceptual overviews on transparency and reproducibility [12]. However, much of the existing guidance focuses on high-level conceptual frameworks or late-stage validation techniques, such as cross-model comparisons or the validation of final outputs against empirical data [13]. There remains a significant gap in the literature regarding practical, hands-on guidance demonstrating exactly how modellers can verify the underlying mechanics of their code during the development phase.

To address the gap, health economists must look beyond traditional HTA practices and adopt foundational quality-assurance (QA) techniques from the field of software engineering. It is common for the health economic modeller to test that their model behaves as they expect but only do it informally, in an ad-hoc way. This includes manually and temporarily changing the model, which can happen interactively, running chunks of code, printing to the console and eyeballing the results. Several authors have noted how formal frameworks and automated checks are lacking [12, 14]. In contrast, software engineers have long automated how they test their code. Their QA is a continuous, structured process integrated throughout the development lifecycle to detect and produce fewer bugs, better code structure, and robust code [15], as well as ensure that the software meets formally defined requirements.

The objective of this tutorial is to demystify automated testing and demonstrate its practical implementation within health economic modelling. Using a realistic Markov cohort model built in R, this article provides a didactic, step-by-step guide to writing, executing, and interpreting automated tests using the widely adopted *testthat* package

[16]. By integrating the framework presented in this tutorial, modellers can establish automated checks that proactively identify and resolve code-level errors before they can influence reimbursement decisions.

2 Hierarchy of Testing

Testing can be performed at different levels of abstraction. Figure 1 illustrates the adapted software engineering *Test Pyramid* framework applied to health economic modelling [17]. The pyramid establishes a structural hierarchy where a high volume of granular, low-level unit tests forms the foundation, supporting a smaller number of more complex, higher-level integration and system tests. Moving upward through the pyramid reflects an increase in the scope and computational cost of the evaluation, alongside a corresponding decrease in execution speed. Integration tests verify that different components or modules work correctly together. For example, testing that the Markov engine correctly processes the transition matrix. System/End-to-End tests verify the fully integrated software product. Because the verification registry proposed in this tutorial runs the complete model pipeline to check summary outputs like QALYs and ICERs, these are technically classified as integration tests rather than classical unit tests.

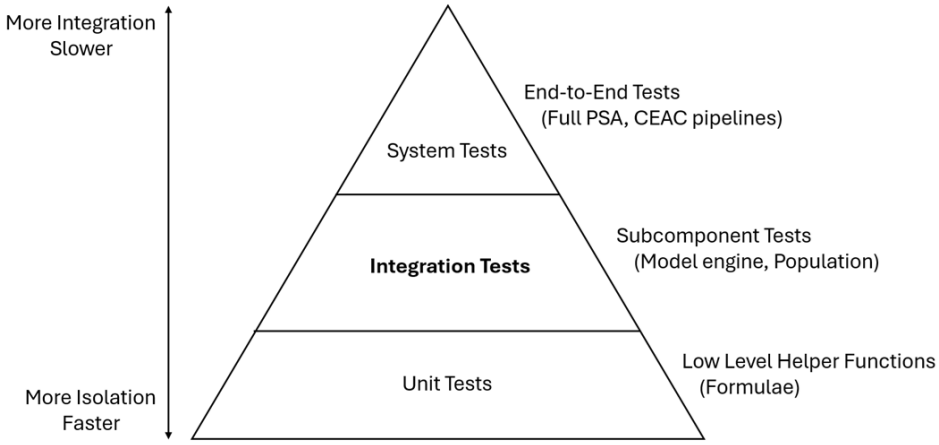


Figure 1: Software engineering Test Pyramid for health economic modelling. CEAC cost-effectiveness acceptability curve; PSA probabilistic sensitivity analysis. 'Integration Tests' is highlighted in bold to reflect the primary focus of the automated checks demonstrated in this tutorial.

2.1 Level 1: Unit Testing (Lowest Level)

In software development, unit testing is the practice of isolating the smallest testable parts of an application referred to as "units", and independently verifying that they function correctly. In the context of a health economic model built in R, a "unit" is typically a single, self-contained function. For example, rather than waiting until the entire Markov trace or PSA is completely generated to check the Incremental Costs, Incremental Effects and the Incremental Cost-Effectiveness Ratio (ICER) for face validity, unit testing evaluates the model's fundamental building blocks, such as the specific function used to calculate a transition probability, apply discounting, or generate a parameter sample.

Note that while some methodologists prefer the term 'Probabilistic Analysis' (PA), we have retained the widely recognised acronym 'PSA' throughout this tutorial to maximise accessibility. Level 1 testing is represented at the base of Figure 1.

2.2 Level 2: Integration Testing (Middle Level)

Level 2 testing is defined as verifying that multiple components (input structures, simulation loops, and summary extractors) interact correctly to produce logically sound results. An HTA example is when executing a model runner function, e.g., `run_model()`, with specific parameters and asserting that the output behaves as expected. The tests are designed to catch silent logic bugs, index shifts, and broken dependencies between clinical inputs and economic outputs. While foundational software engineering literature emphasizes the isolation of pure functions via unit testing, health economic models frequently require this broader integration approach due to the absence of a simple 'test oracle' (an absolute known output for complex multi-state traces). Consequently, rather than verifying isolated lines of code, the automated checks presented in this tutorial primarily operate as integration tests. They confirm that the integration of input parameters, transition matrices, and simulation engines preserves foundational economic logic—serving as a rigorous engine for internal model verification. This is shown in the middle layer of Figure 1.

2.3 Level 3: System or End-to-End Testing (High Level)

System, or end-to-end testing can be defined as executing the entire decision-analytic pipeline from raw data ingestion to final policy outputs. In an HTA context, an example would be running a 10,000 simulation Probabilistic Sensitivity Analysis (PSA) to verify that the pipeline executes without memory leaks, handles missing data gracefully, and successfully outputs an Incremental Cost-Effectiveness Ratio (ICER). Level 3 testing is represented at the top of Figure 1.

3 The *testthat* Package

The *testthat* package has emerged as the definitive standard for automated testing within the R ecosystem [16]. For health economists, *testthat* provides a structured, highly readable syntax built around plain-English naming conventions. This reduces the barrier to entry, making test scripts almost as readable as the conceptual methodologies they are verifying To setup a package to use *testthat* run the following R code

```
usethis::use_testthat(3)
```

The double colon (`::`) acts as a namespace operator in R. It explicitly calls the `use_testthat()` function directly from the *usethis* package, which prevents naming conflicts and avoids the need to load the entire package into the environment. The folder `tests/testthat` will be created and added to the `DESCRIPTION` file. The file `tests/testthat.R` is then created, which runs the tests when the R check command is run. As functions are defined in a package, the corresponding tests are added to `.R` files in `tests/testthat/`. Filenames with tests in must start with `test`. The *usethis* package has some convenience functions, including `use_test(name)` which creates and opens a test file called `tests/testthat/test-name.R`.

The automated checks written in this syntax are called *expectations*, which form the most basic unit of using *testthat*. Readers familiar with Microsoft Excel VBA may recognise these functions as being conceptually identical to assertions (e.g., `Debug.Assert` in VBA). All expectations start with `expect_` (e.g., `expect_equal()`, `expect_error()`, `expect_type()`), have the actual and expected results as arguments, and they throw an error if these two things don't agree. Collections of related expectations are grouped together in to what *testthat* calls tests. A group is wrapped by the `test_that("<description>", ...)`. A file then hold multiple related tests. Furthermore, *testthat* allows modellers to bundle dozens or hundreds of individual unit tests into a cohesive "test suite," which can be executed seamlessly with a single command, providing an immediate, visual summary of which model components passed and which require debugging. For an in depth reference of the *testthat* package see [15].

While a modeller might understandably ask why simple mathematical operators (e.g., using `== 0` or `<`) cannot be used directly in the script in place of complex functions, these ad-hoc approaches present significant limitations for rigorous QA. A simple mathematical evaluation might return a generic FALSE or fail silently within the console, providing no context regarding the source of the error. Furthermore, manual checks require the modeller to actively read and interpret the console output every time the script is executed. In contrast, *testthat* expectations (such as `expect_equal()` and `expect_lt()`, which stands for 'expect less than') are explicitly designed to act as automated diagnostic tools. When an expectation fails, the package provides a structured, highly readable error message indicating exactly which component failed and the discrepancy between the actual and expected values.

4 Health Economic Modelling Test Suite

To ensure the foundational accuracy of the underlying computation and prevent silent error propagation, modellers can implement a standardized suite of automated unit tests. Table 1 presents an example of practical verification tests, extreme-value checks, and logical assertions adapted from established HTA quality assurance frameworks. Each test is assigned a unique identifier and belongs to a set of tests, e.g., related to cost or QALY calculations, or population counts. For each test, a test condition is defined which specifies the scenario input parameter of interest, e.g., setting the discount rate to 0. An expected result is obtained from running the model, e.g., with no discounting (and assuming the utility of all living health states is set to 1), then the total QALYs are the same as the life expectancy.

Specifically, these test scenarios operationalize the internal consistency and extreme-value checks introduced by McCabe and Dixon (2000) [18], the comprehensive "black-box" verification criteria formalized by Tappenden and Chilcott (2014) [19], the automated decision-analytic checking principles outlined by Dasbach and Elbasha (2017) [20], and the technical verification standards reinforced by the CHEERS statement [21]. The following section will practically demonstrate how to translate these conceptual checks into (automated) *testthat* expectations.

ID	Group	Test Condition	Expected Result
T01	Population	Set relative treatment effects (odds ratio, hazard ratio, risk ratio) = 1	All treatments produce equal estimates of total LYGs and total QALYs
T02	Population	Set mortality rates (or corresponding transition probabilities) = 0	LE = time horizon (assuming a fixed, finite time horizon to avoid an infinite loop)
T03	Population	Set mortality rates (or corresponding transition probabilities) to produce a probability of 1	LE = 1 cycle (or 0.5 depending on within-cycle correction implementation)
T04	QALY	Set discount rate = 0 (assuming utility = 1 for all living health states)	Total QALYs equal LE
T05	QALY	Set all living health state utility parameters = 1	QALY gains exactly equal LYGs
T06	QALY	Apply standard positive discount rate vs. 0 rate	Discounted QALYs $<$ undiscounted QALYs $\forall$ treatments
T07	QALY	Set QALY discount rate = ∞	QALY gains after time 0 equal zero
T08	Cost	Set intervention costs = 0	ICER is reduced†
T09	Cost	Increase intervention costs	ICER is increased†
T10	Cost	Set cost discount rate = 0	Total costs exactly equal the raw arithmetic sum of unadjusted state costs $\forall$ treatments
T11	Cost	Set cost discount rate = ∞	Costs after time 0 equal zero
T12	Input	Sample parameters from probabilistic distributions	Range of sampled values does not violate distribution boundaries, NaNs, or critical numerical precision errors

Table 1: Example suite of verification tests (registry) for health economic models. QALY quality-adjusted life-years; LYG life-years gained; ICER incremental cost-effectiveness ratio; LE life expectancy. †Assuming a controlled test environment where the baseline Incremental Effects are known to be strictly positive. If evaluating a scenario with negative incremental health effects, the verification logic must isolate Incremental Costs (IC) instead. All checks in this registry are classified as integration or system-level tests, rather than classical unit tests of isolated functions.

5 Case Study: A Discrete-Time Markov Model

5.1 Model Overview

A 3-state Sick-Sicker-Dead-type cohort model is considered. Figure 2 shows a schematic representation of the cohort Markov model.

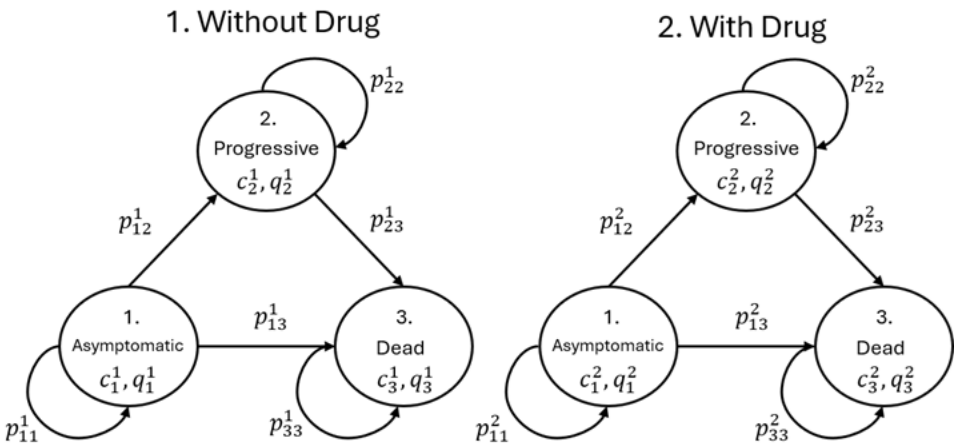


Figure 2: Markov model diagram for Without Drug and With Drug scenarios.

Define the baseline input values for the 3-state model with two intervention scenarios of 'Without Drug' and 'With Drug', indexed by $s = 1$ and 2, respectively. The drug unit cost is 400 monetary units. The parameters p^s_{xy} are the transition probabilities from state x to y for scenario s, and c^s_x and q^s_x are the costs and health measures (utilities) for state x and scenario s. The state occupancy costs and utilities are

$$C^1 = \begin{pmatrix} 100 \\ 200 \\ 0 \end{pmatrix}, \quad C^2 = \begin{pmatrix} 500 \\ 200 \\ 0 \end{pmatrix}, \quad Q^s = \begin{pmatrix} 0.9 \\ 0.6 \\ 0 \end{pmatrix},$$

and the transition probabilities are

$$P^1 = \begin{pmatrix} 0.75 & 0.2 & 0.05 \\ 0 & 0.9 & 0.1 \\ 0 & 0 & 1 \end{pmatrix}, \quad P^2 = \begin{pmatrix} 0.85 & 0.1 & 0.05 \\ 0 & 0.95 & 0.05 \\ 0 & 0 & 1 \end{pmatrix}$$

Define a cohort of size $n = 1000$ to all start in state $x = 1$ (Asymptomatic) at time $t = 0$. We will simulate to the time horizon for $n_{\text{cycles}} = 15$ cycles.

5.2 Model Architecture

In software engineering, a *monolithic* architecture describes a specific design approach where an application is built as a single, unified codebase. In such systems, the core mathematical calculations (business logic), the user-facing scripts or dashboards (user interface), and the processes for loading parameter inputs (data access layers) are tightly interwoven. To effectively implement automated verification using the `testthat` package, the underlying R code cannot be written as a single monolithic script. Instead, the model's architecture must embrace the software engineering principle of "separation of

concerns," dividing the computational pipeline into discrete, highly modular components. Modularity is what fundamentally makes a model "testable."

A common temptation for analysts transitioning from spreadsheets is to define all parameters globally and execute calculations in a single, top-to-bottom monolithic script. However, relying on the global environment makes code brittle and inherently difficult to follow and verify. If parameters are defined globally, testing a single boundary condition requires permanently overwriting the global state. The changes silently contaminate all subsequent analyses. By encapsulating the model within self-contained functions (such as the `run_model()` wrapper) and isolating the inputs, the model becomes fully testable. Functional architecture allows the modeller to inject test values and evaluate specific subcomponents in total isolation, ensuring that one automated check does not inadvertently alter the data required for the next.

For the cohort Markov model, the architecture is explicitly separated into three distinct domains. First, the *input parameters* (such as transition matrices, cost vectors, and utility weights) are isolated into independent lists. Second, the *simulation engine* (the `ce_markov()` function) accepts the input parameters, executes the matrix multiplication across the specified time horizon, and returns a single object with all outputs. Third, *helper functions* simplify the code with small, reusable functions that abstract away common or repetitive tasks in the whole modelling pipeline so that the main logic stays clean.

By compartmentalizing the code in this manner, modellers can isolate and test individual units of logic. For example, we can pass an extreme boundary value into the simulation engine and isolate the output extractor to verify that a specific mathematical expectations hold (for instance, confirming that setting intervention costs to zero strictly reduces the ICER, as outlined in test T08), without having to run an entire PSA pipeline. To facilitate the automated test suite detailed in Section 6, we define a standardized baseline input list, `base_params`, which serves as the universal reference point for all subsequent logic checks.

6 R Test Code Examples

In this section, we will step through how to use the *testthat* package in R to automate verification tests of a cost-effectiveness analysis using the defined cohort Markov model.

This tutorial used R (v 4.6.1) and the *testthat* package (v 3.3.2). To provide a fully reproducible, hands-on example without restricting the conceptual focus to a single model type, we have provided a minimal executable model and the complete test suite in the accompanying GitHub repository (URL). Readers are encouraged to download this repository to run the end-to-end tests locally.

6.1 Input Parameters

The model definition in Section 5.1, can be translated into R code input values in a list format as follows.

```
# basic test data
t_names <- c("without_drug", "with_drug") # treatments
s_names <- c("Asymptomatic", "Progressive", "Dead") # states
```

```
# base CE model input parameters
base_params <- list(
  # population at time 0 all in Asymptomatic state
  start_pop = c(1000, 0, 0),
  # transition probability matrix
  p_matrix = array(
    c(0.75, 0, 0, 0.2, 0.9, 0, 0.05, 0.1, 1,
      0.85, 0, 0, 0.1, 0.95, 0, 0.05, 0.05, 1),
    dim = c(3,3,2),
    dimnames = list(s_names, s_names, t_names)),
# state costs
  state_c_matrix = matrix(
    c(100, 200, 0,
      500, 200, 0),
    nrow = 2, ncol = 3, byrow = TRUE,
    dimnames = list(t_names, s_names)
  ),
  # state utilities
  state_q_matrix = matrix(
    c(0.9, 0.6, 0,
      0.9, 0.6, 0),
    nrow = 2, ncol = 3, byrow = TRUE,
    dimnames = list(t_names, s_names)
  ),
  n_cycles = 15, # time horizon
  discount_rate = 0.035
)
```

The object `base_params` serves as the baseline set of inputs which can be selectively modified for each test.

Throughout this tutorial, functions such as `run_model()`, `get_costs()`, and `get_qalys()` are used to represent the model engine and its extraction helpers. These are deliberate abstractions designed to illustrate the testing interface. In practice, the internal implementation of these functions will be uniquely defined by the modeller based on the specific architecture of their project (e.g., a Markov cohort model versus a microsimulation). The automated testing framework presented here evaluates the inputs and outputs at this interface layer, making it agnostic to the underlying mechanics of the model itself.

6.2 Helper Functions

A convenient idea is to use *setters* or *mutators*, and *getters* or *accessors* to add a layer of abstraction to interact with the models. This approach borrows from Object-Oriented Programming (OOP) principles, specifically encapsulation, which shields the user from the complex internal structure of the model object. For a comprehensive overview of how these OOP paradigms apply to R programming, we direct readers to Wickham (2019) [22]. Although creating these may initially seem like an unnecessary extra step, these simple helper functions are highly useful for maintaining a robust test suite over time. They decouple the automated test scripts from the complex internal data structures of the model.

A setter can be used to safely modify a specific parameter within the `base_params`

list without redefining the whole object and without risking the accidental overwriting of adjacent data. Similarly, a getter can easily extract targeted outputs (like QALYs, costs, or Life Expectancy) from the results of a model run. Ultimately, if the model's underlying data structure is updated in the future, the modeller only needs to update the single helper function rather than rewriting dozens of broken unit tests. The inner workings of the setters and getters are particular to a model but all the user needs to know is `set_discount_rate()` or `get_qalys()` and not the structure of the object that is passed to or returned from that model.

```
# setter helper function
set_discount_rate <- function(input, discount_rate) {
  input$discount_rate <- discount_rate
  input
}
```

and to return the total QALYs after a model run the following code.

```
# getter helper function
get_qalys <- function(res) {
  res$total_QALYs
}
```

The inner workings of the setters and getters are particular to a model but all the user needs to know is `set_discount_rate()` or `get_qalys()` and not the structure of the object that is passed to or returned from that model.

As an extension to this idea, to facilitate the following tests, define a *wrapper* function `run_model()`, allowing us to pass the default dataset but override specific parameters we want to test (like the discount rate). One way of thinking of this approach is as a type of setter function as part of the call to `ce_markov()`.

```
# CE model convenience  wrapper function
run_model <- function(input_data, ...) {
  # merge default input_data with manual overrides (...)
  updates <- list(...)

  # replace with new values
  args_list <- modifyList(input_data, updates)

  # call the main model function
  do.call(ce_markov, args_list)
}
```

Using `...` (ellipses) allows us to write clean tests such as `run_model(base_data, discount_rate = 0)` without having to manually reconstruct the entire input list every time. In the next sections, we will give examples of three types of verification checks.

6.3 Discounting Tests

A fundamental property of health economic models is that the present value (PV) of a future value (FV) decreases as time t increases further into the future and the discounting increases. For example, the present value of a QALY at time t is given by the following.

$$q_{PV} = \frac{q_{FV}}{(1+r)^t}$$

Testing the model code when there is discounting in the model is crucial, since it will modify the total costs and health values and the decisions. The verification tests that correspond to the effect of discounting in Table 1 are T04, T06, T07, (for QALYs), T10, T11 (for costs). To demonstrate test T06 in Table 1, which verifies the expectation that applying a standard positive discount rate will result in strictly lower total QALYs compared to an undiscounted 0% rate, the test case can be broken down into three steps:

- Step 1: Run model with discount rate $r = 0\%$, i.e. $q^s_{PV} = q^s_{FV}$ (undiscounted) for scenarios $s = 1, 2$.
- Step 2: Run model with discount rate $r = 3.5\%$ (discounted).
- Step 3: Check expectation that the discounted QALYs are strictly less that the undiscounted QALYs for each treatment, $q^s_{PV}(r = 3.5\%) < q^s_{PV}(r = 0\%)$.

Below, the code needed to perform unit tests to verify discounting logic is detailed. The code block begins with the `test_that()` function, which requires a descriptive character string as its first argument. While the specific wording is flexible, we highly recommend prefixing this string with the unique identifier from the Test Registry (e.g., 'T07: ...'). Doing so ensures strict traceability; if the code fails, the console will explicitly flag that requirement T06 is broken, streamlining the auditing process. The Markov model is run with with 3.5% discounting and assign the results to `results_discounted`, and with 0% discounting and assign the results to `results_undiscounted`. This results in how to 'get' the QALYs from these and compare the scenarios with and without drugs in expectation statements. Specifically, the `expect_lt()` ('expect less than') function is used to assert that the discounted values are strictly lower.

```
test_that("T06: Discounted QALYs are lower than undiscounted QALYs", {
  # run model with standard discounting (3.5%)
  results_discounted <- with(base_params,
   ce_markov(start_pop, p_matrix, state_c_matrix,
   state_q_matrix, n_cycles, discount_rate = 0.035))

  # run model with NO discounting (0%)
  results_undiscounted <- with(base_params,
   ce_markov(start_pop, p_matrix, state_c_matrix,
   state_q_matrix, n_cycles, discount_rate = 0.0))

  # alternatively:
  # results_discounted <-
  #  run_model(base_params, discount_rate = 0.035)
  # results_undiscounted <-
  #  run_model(base_params, discount_rate = 0.00)

  # extract the total QALYs using getter
  # could use direct list access instead
  qaly_discounted <- get_qalys(results_discounted)
  qaly_undiscounted <- get_qalys(results_undiscounted)

  # expectation: discounted must be less than undiscounted value
  expect_lt(qaly_discounted["without_drug"],
            qaly_undiscounted["without_drug"])
```

```
  expect_lt(qaly_discounted["with_drug"],
            qaly_undiscounted["with_drug"])
})
```

Next, test T07 from Table 1 is implemented in R, namely that the edge case of a sufficiently large discounting rate r results in the costs and QALYs reducing to 0, i.e. $q_{PV} \to 0$ as $r \to \infty$. Because the test depends on convergence, the `tolerance` argument is used in `expect_equal()` which allows the expectation to return `TRUE` if the expected and actual values are within some margin of error. In the remaining examples, the `run_model()` helper function is used.

```
test_that("T07 & T11: High discounting reduces future value to 0", {
  # Run full model with extreme discount
  res_high_disc <- run_model(base_params, discount_rate = 1000)

  # Run a baseline model for only 1 cycle (time 0)
  res_cycle_zero <- run_model(base_params, n_cycles = 1)

  # Check Costs (T11)
  expect_equal(
    get_costs(res_high_disc),
    get_costs(res_cycle_zero),
    tolerance = 0.01,
    label = "T11: High discount total costs equal cycle 0 costs"
  )

  # Check QALYs (T07)
  expect_equal(
    get_qalys(res_high_disc),
    get_qalys(res_cycle_zero),
    tolerance = 0.01,
    label = "T07: High discount total QALYs equal cycle 0 QALYs"
  )
})
```

While an infinite bound (`Inf`) is mathematically ideal to capture extreme values, an arbitrarily large finite number is recommended in these templates. This is a pragmatic choice to ensure the test code remains generalisable and computationally stable when adapted for users' specific downstream operations (e.g., numerical optimisation), which often fail when encountering strictly infinite values.

6.4 Unit Costs and Utilities Tests

Tests for unit costs and utilities are often boundary value type tests that assign zeros and ones, and are simple calculations that could often be performed without even running the model and can be thought of as "sanity checks." Test T08 in Table 1 is implemented and evaluates the condition where the specific cost of the intervention is removed. To simulate this boundary condition, the unit costs for the 'With Drug' scenario are modified to exactly equal the baseline 'Without Drug' costs, effectively reducing the intervention cost to zero.

$$C^2 = C^1$$

Provided the test baseline is controlled to produce positive incremental health effects, verifying that the ICER for this zero-intervention-cost case is necessarily reduced from the baseline ICER aligns perfectly with established extreme-value verification frameworks.

```
test_that("T08: Zero intervention cost strictly reduces ICER", {

  # Temporary wrapper to isolate intervention cost
  run_with_intervention_cost <- function(c_int) {
    temp_params <- base_params
    temp_params$state_c_matrix["with_drug", "Asymptomatic"] <-
      temp_params$state_c_matrix["without_drug", "Asymptomatic"] + c_int

    run_model(temp_params)
  }

  # Run model with a standard positive intervention cost
  res_base <- run_model(base_params)

  # Run model with zero intervention cost
  res_no_int <- run_with_intervention_cost(c_int = 0)

  # extract ICERs
  res_no_int_icer <- get_icer(res_no_int)
  res_base_icer <- get_icer(res_base)

  # extract baseline Incremental Effects (IE)
  # to handle ICER directionality
  base_qalys <- get_qalys(res_base)
  res_base_ie <- unname(base_qalys["with_drug"] -
                  base_qalys["without_drug"])

  # verify ICER directional change based on the sign of IE
  # skip the rest of this block if assumption fails
  testthat::skip_if_not(
    res_base_ie > 0,
    "Baseline incremental QALYs are <= 0.
     Skipping ICER directionality tests."
  )

  expect_lt(res_no_int_icer, res_base_icer)
})
```

6.5 Population Tests

The transition matrix logic can be verified by forcing extreme mortality risks to modify life expectancy. Tests such as these are types of boundary value tests. It is not uncommon for row sums to drift away from 1 due to rounding errors or copy-paste mistakes. Automated testing catches an issue instantly before the trace is even run. From Table 1, population focused tests correspond to T01, T02, T03. A demonstration of T03 is performed with the test case to set probability of death to 1 (i.e. 100%) immediately for everyone. The life expectancy is expected to be exactly 1 cycle. The state transition probability matrix

for the intervention scenarios $s = 1, 2$ is

$$P^s = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{pmatrix}$$

In the case of time-dependent transition probabilities, because the 'Dead' state is an absorbing state, simply setting $P^s(t = 1)$ would have the same effect. To implement the test case in the code, first take the shape of the transition probability matrix from `p_matrix` and then assign all probabilities to zero. We then assign a probability of one exclusively to the transitions going to the 'Dead' state. Utilising the named dimension ('Dead') rather than a numeric column index ensures that the test code remains highly readable and self-documenting.

```
test_that("T03: Immediate mortality results in LE of 1", {
  # modify p_matrix to force death
  dead_p_matrix <- base_params$p_matrix
  dead_p_matrix[] <- 0
  dead_p_matrix[, "Dead", ] <- 1 # all transitions go to Dead

  res_death <- run_model(base_params, p_matrix = dead_p_matrix)
  res_le <- get_le(res_death)

  # is Life Expectancy for all scenarios = 1 cycle
  expect_equal(res_le, rep(1, length(res_le)))
})
```

When designing checks for cumulative outcomes such as life expectancy, total QALYs, or total costs, standard discrete-time Markov models often apply a half-cycle (or within-cycle) correction to account for transitions occurring continuously throughout a cycle rather than abruptly at the end. If such a correction is implemented in the model engine, the expected results in the automated tests must be adjusted to match. For example, in a model applying a standard half-cycle correction, setting mortality to 100% in the first cycle (test T03) will conceptually result in a life expectancy of 0.5 cycles rather than exactly 1.0 cycle.

7 Automating the Process

In Wickham and Bryan, [15] they define three different scales at which test can be run. The case study example above can fit into the first, called *micro-iteration*, which is the interactive phase useful for refining and debugging. These tests do not have to live inside an R package but can be run directly.

The true power of unit testing comes from automation. The entire test suite can be run all at once to ensure that new changes to the model do not break existing logic, a so-called *macro-iteration* scale of testing [15].

Practically, all tests in the package can be run with `devtools::test()`. The typical output will be a report in the console with testing details and a summary similar to the following. For all tests passing

```
[ FAIL 0 | WARN 0 | SKIP 0 | PASS 3 ]
```

Where `PASS` means the model logic is correct and the expectation corresponds to the model output. `FAIL` means that something is broken. To illustrate the value of the explicit naming conventions recommended in Section 6.3, consider a scenario where a modeller inadvertently applies the discount rate incorrectly, causing the discounted QALYs to exceed the undiscounted QALYs. The automated test suite will immediately halt and print a specific error trace to the console e.g.

```
Failure (test-discounting.R:14:3): T06: Discounted QALYs are
   lower than undiscounted QALYs
qaly_discounted["without_drug"] (‘actual’) not less than qaly
   _undiscounted["without_drug"] (‘expected‘).

[ FAIL 1 | WARN 0 | SKIP 0 | PASS 2 ]
```

Because the test was explicitly prefixed with the Test Registry ID, the error message tells the modeller exactly which logic check failed (‘T06’), which file the error occurred in, and the exact mathematical discrepancy between the actual and expected values. Any identified failed test can then be investigated and fixed so that it passes.

8 Discussion

8.1 Implications and Comparison with Existing Practices

Historically, health economic models have been predominantly developed in spreadsheet software, where validation relies heavily on manual cell-by-cell checking, double-programming, and ad-hoc peer review. While these traditional verification methods are well-documented, they are inherently prone to human error and are difficult to scale. In recent years, collaborative initiatives, such as R in HTA [23] and the Decision Analysis in R for Technologies in Health (DARTH) workgroup, have pioneered the transition toward open-source programming in R, highlighting its capacity for superior computational efficiency and transparency. However, while the literature robustly supports the *building* of models in R, the specific operationalisation of automated, code-based verification within HTA has remained relatively underexplored.

This tutorial demonstrates how software engineering practices can bridge this gap. The integration of automated testing provides a mathematically rigorous, auditable alternative to manual spreadsheet debugging. For example, by catching logic failures at the unit level before they propagate into final incremental cost-effectiveness ratio (ICER) calculations, automated test suites ensure long-term methodological robustness. Furthermore, a comprehensive test suite facilitates seamless model adaptations; modellers can confidently add new health states or evaluate new comparators, knowing that automated checks will immediately flag any unintended consequences to existing code. Ultimately, adopting these rigorous verification practices has significant implications for HTA policy, potentially accelerating peer and agency review processes (such as those by NICE or the PBAC) by presenting evaluation committees with a transparent, computationally verified framework.

8.2 Limitations

Despite its clear methodological advantages, the widespread adoption of automated testing in health economics faces practical limitations. First, economic evaluation itself demands a substantial learning curve, requiring modellers to master a complex intersection of epidemiology, statistical methodology, and decision theory. The transition from visually intuitive spreadsheet environments to abstract, script-based testing frameworks requires a fundamental shift in technical competencies. The intention of outlining this framework is not to mandate that health economists must become software engineers, but rather to demonstrate how adopting a targeted subset of engineering practices protects the rigorous methodological work already undertaken.

In addition, it is crucial to recognise the conceptual boundaries of the tests described in this tutorial. Passing an automated test suite guarantees strict code verification (internal consistency), but does not equate to clinical validation (external validity). As outlined by the ISPOR-SMDM Modeling Good Research Practices Task Force, verifying the accuracy of mathematical coding is an essential prerequisite, but it must be accompanied by broader validation efforts, such as face validity, cross-validity, and the comparison of simulated results against real-world epidemiological data [9]. Automated testing confirms that the model is built right, but external clinical validation is still required to ensure that the right model was built.

8.3 Future Directions in Model Verification

The implementation of standard unit testing serves as a gateway to more advanced quality assurance workflows. A natural next step is the adoption of Continuous Integration (CI) pipelines hosted on version-control platforms such as GitHub. CI frameworks automatically execute the entire *testthat* suite every time a code modification is pushed to the repository, preventing broken code from merging into the main project. Modellers can also expand their testing toolkits by using packages like *checkmate*, which provides fast, optimised assertions designed for complex R objects [24], or implement tools such as *covr* to formally evaluate overall test coverage [25]. Finally, as modellers build increasingly computationally intensive simulations that rely on compiled languages (such as integrating C++ via the *Rcpp* package), future methodological guidance must expand to encompass low-level unit testing frameworks designed specifically for these advanced programming environments.

As computational literacy within the field matures, model building should increasingly begin with structured requirements capture and formal documentation. Doing so enables the embrace of Test-Driven Development (TDD) [26]. In a TDD paradigm, modellers write unit tests based on the requirements of HTA protocols before writing the underlying model functions. This approach forces modellers to define exact clinical expectations and boundary conditions at the inception of the project, ensuring that the model's architecture is guided by rigorous verification criteria from the outset.

A promising means of lowering the learning curve for these advanced techniques is the use of Large Language Model (LLM) assistants to accelerate the authoring of *testthat* code. Given a plain-language description of an intended check and the relevant function signatures, an LLM can produce a syntactically correct test block in seconds, effectively acting as a first-draft author that the domain expert then reviews and validates. This workflow, along with considerations for tool selection, is described in detail in Appendix B. Importantly, however, AI assistance does not remove the need for expert judgement: a

generated test that inverts the intended inequality (e.g., asserting discounted QALYs are *greater* than undiscounted QALYs) is syntactically indistinguishable from a correct one and will pass silently unless the modeller reads and understands what has been committed. AI tools are therefore best positioned as efficiency aids within a human-led review process, not as autonomous verification.

Although adopting a full software development lifecycle may seem overly rigorous for models intended for a single use or a specific, one-off policy appraisal, the core tenets of software engineering remain essential. All health economic models must be fully reproducible and transparent, regardless of whether they are intended for long-term reuse or a single publication. A model used even once to inform critical healthcare resource allocation must be demonstrably free of logical and mathematical errors. The adoption of automated testing provides an auditable, reproducible record that these foundational standards have been met, without strictly requiring the modeller to maintain the code as a formal software package.

Looking ahead, the health economics community would benefit from establishing a universal, standardised suite of automated verification tests, ideally codified and endorsed by professional bodies such as ISPOR or SMDM. Having a centralised, open-source library of baseline HTA tests would unify global verification standards. To understand the level of rigour required, health economists can look to other safety-critical fields. For example, integration testing in the automotive industry utilizes highly regulated compliance frameworks such as Automotive SPICE (A-SPICE) and the Federal Motor Vehicle Safety Standards (FMVSS). Managed by the National Highway Traffic Safety Administration (NHTSA), these strict frameworks dictate how integration testing must be designed, executed, and documented to govern the safety performance requirements for motor vehicles. [27, 28]. Ultimately, the safety-critical compliance frameworks seen in other high-stakes industries, such as A-SPICE and FMVSS, can act as a roadmap for standardising integration testing in HTA.

9 Conclusion

As health economic models grow in structural and computational complexity, and the field continues its paradigm shift toward open-source languages like R, there is a critical need to adopt foundational quality-assurance techniques from software engineering. A demonstration has been given of how the integration of automated testing, specifically utilizing the *testthat* package, provides a rigorous, hands-on framework for internal model verification. By systematically verifying mathematical logic, transition dynamics, and extreme-value boundaries at the component level, modellers can proactively identify and resolve silent computational errors before they propagate through the model. Ultimately, embracing these testing workflows is essential to ensure computational accuracy, improve transparency, and prevent consequential errors in final HTA and reimbursement decisions.

Code Availability Statement

All R code, test scripts, and the accompanying case study generated for this tutorial are publicly available through an open-source GitHub repository at `https://github.com/validate-hta/testhta` to facilitate immediate adoption and replication.

Funding

The authors declare that no grants from any funding agency in the public, commercial, or not-for-profit sectors were received to support the research, authorship, and/or publication of this tutorial.

Conflicts of Interest

Daniel Gallardo is an employee of ICON Clinical Research. Nathan Green and Felicity Lamrock declare that they have no potential conflicts of interest relevant to the content of this manuscript.

References

[1] Demi Radeva, Gareth Hopkin, Elias Mossialos, John Borrill, Leeza Osipenko, and Huseyin Naci. Assessment of technical errors and validation processes in economic models submitted by the company for nice technology appraisals. *International Journal of Technology Assessment in Health Care*, 36:311–316, 2020. doi: 10.1017/s0266462320000422.

[2] S. Montagnoli et al. Patient-level simulation models in cancer care: a systematic review. *Frontiers in Public Health*, 12:1335300, 2024. doi: 10.3389/fpubh.2025.1335300.

[3] Hawre Jalal, Petros Pechlivanoglou, Eline Krijkamp, Fernando Alarid-Escudero, Eva Enns, and M G Myriam Hunink. An overview of R in health decision sciences. *Medical Decision Making*, 37(6):735–746, 2017. doi: 10.1177/0272989X16686559.

[4] Nathan Green, Felicity Lamrock, Nichola Naylor, Jack Williams, and Andrew Briggs. Health economic evaluation using markov models in R for microsoft excel users: A tutorial. *PharmacoEconomics*, 41:5–19, 2022. doi: 10.1007/s40273-022-01199-7.

[5] Nichola R. Naylor, Jack Williams, Nathan Green, Felicity Lamrock, and Andrew Briggs. Extensions of health economic evaluations in R for microsoft excel users: A tutorial for incorporating heterogeneity and conducting value of information analyses. *PharmacoEconomics*, 41:21–32, 2022. doi: 10.1007/s40273-022-01203-0.

[6] R. Smith, W. Mohammed, and P. Schneider. Packaging cost-effectiveness models in R: A tutorial. *Wellcome Open Research*, 8:419, 2023. doi: 10.12688/wellcomeopenres.19656.1.

[7] Salah Ghabri, Matt Stevenson, Jörgen Möller, and J. Jaime Caro. Trusting the results of model-based economic analyses: Is there a pragmatic validation solution? *PharmacoEconomics*, 37:1–6, 2018. doi: 10.1007/s40273-018-0711-9.

[8] Mark Roberts, Louise B. Russell, A. David Paltiel, Michael Chambers, Phil McEwan, and Murray Krahn. Conceptualizing a model: A report of the ISPOR-SMDM modeling good research practices task force-2. *Value in Health*, 15(6):804–811, 2012. doi: 10.1016/j.jval.2012.06.016.

[9] David M. Eddy, William Hollingworth, J. Jaime Caro, Joel Tsevat, Kathryn M. McDonald, and John B. Wong. Model transparency and validation: A report of the ISPOR-SMDM modeling good research practices task force-7. *Value in Health*, 15 (6):843–850, 2012. doi: 10.1016/j.jval.2012.04.012.

[10] J. Jaime Caro, Andrew H. Briggs, Uwe Siebert, and Karen M. Kuntz. Modeling good research practices—overview: A report of the ISPOR-SMDM modeling good research practices task force-1. *Value in Health*, 15(6):796–803, 2012. doi: 10.1016/j.jval.2012.06.012.

[11] Zorginstituut Nederland. Richtlijn voor het bouwen van kosteneffectiviteitsmodellen in R [guideline for building cost-effectiveness models in R]. Guideline, Zorginstituut Nederland, Diemen, The Netherlands, 12 2022.

[12] F. Alarid-Escudero et al. A need for change! A coding framework for improving transparency in decision modeling. *PharmacoEconomics*, 37(11):1329–1339, 2019.

[13] R. A. Smith, Y. Samyshkin, W. Mohammed, et al. assertHE: an R package to improve quality assurance of HTA models. *Wellcome Open Research*, 9:701, 2024. doi: 10.12688/wellcomeopenres.23180.1.

[14] Nasuh C. Büyükkaramikli, Maureen P. M. H. Rutten-van Mölken, Johan L. Severens, and Maiwenn Al. TECH-VER: A verification checklist to reduce errors in models and improve their credibility. *PharmacoEconomics*, 37:1391–1408, 2019. doi: 10.1007/s40273-019-00844-y.

[15] Hadley Wickham and Jennifer Bryan. *R Packages: Organize, Test, Document, and Share Your Code*. O'Reilly Media, 2nd edition, 2023.

[16] Hadley Wickham. testthat: Get started with testing. *The R Journal*, 3:5–10, 2011. URL `https://journal.r-project.org/articles/RJ-2011-002/`.

[17] Mike Cohn. *Succeeding with Agile: Software Development Using Scrum*. Addison-Wesley Professional, Boston, MA, 2009. ISBN 978-0321579362.

[18] C. McCabe and S. Dixon. Testing the validity of cost-effectiveness models. *PharmacoEconomics*, 17(5):501–513, 2000. doi: 10.2165/00019053-200017050-00007.

[19] P. Tappenden and J. B. Chilcott. Avoiding and identifying errors and other threats to the credibility of health economic models. *PharmacoEconomics*, 32:967–979, 2014. doi: 10.1007/s40273-014-0186-2.

[20] E. J. Dasbach and E. H. Elbasha. Verification of decision-analytic models for health economic evaluations: An overview. *PharmacoEconomics*, 35:673–683, 2017. doi: 10.1007/s40273-017-0508-2.

[21] D. Husereau et al. Consolidated health economic evaluation reporting standards (CHEERS) statement. *European Journal of Health Economics*, 14(3):367–372, 2013. doi: 10.1007/s10198-013-0471-6.

[22] Hadley Wickham. *Advanced R*, volume XXXIII. Chapman & Hall/CRC, 2014. ISBN 9780874216561. doi: 10.1007/s13398-014-0173-7. 2. URL http://www.americanbanker.com/issues/179_124/which-city-is-the-next-big-fintech-hub-new-york-stakes-its-claim-1068345-1.html%5Cnhttp://www.ncbi.nlm.nih.gov/pubmed/15003161%5Cnhttp://cid.oxfordjournals.org/lookup/doi/10.1093/cid/cir991%5Cnhttp://www.scielo.

[23] Gianluca Baio, Howard Thom, and Petros Pechlivanoglou, editors. *R for Health Technology Assessment*. Chapman & Hall/CRC Biostatistics Series. Chapman and Hall/CRC, Boca Raton, 2025. ISBN 9780367468910. doi: 10.1201/9781003031819.

[24] Michel Lang. checkmate: Fast argument checks for defensive R programming. *The R Journal*, 9(1):437–445, 2017. doi: 10.32614/RJ-2017-028.

[25] Jim Hester. *covr: Test Coverage for Packages*, 2023. URL https://covr.r-lib.org. R package version 3.6.4, https://github.com/r-lib/covr.

[26] Kent Beck. *Test-Driven Development: By Example*. Addison-Wesley Professional, 2002. ISBN 9780321146533.

[27] VDA QMC. *Automotive SPICE Process Reference Model / Process Assessment Model v4.0*. Verband der Automobilindustrie (VDA), 2023. Available at https://www.vda-qmc.de/.

[28] National Highway Traffic Safety Administration. Federal motor vehicle safety standards. 49 C.F.R. § 571, 2026. U.S. Department of Transportation.

[29] Jim Hester, Lionel Henry, Kirill Müller, Kevin Ushey, Hadley Wickham, and Winston Chang. *withr: Run Code 'With' Temporarily Modified Global State*, 2026. URL https://withr.r-lib.org. R package version 3.0.3.

A Appendix

A.1 Custom Expectations

Custom helper functions are highly effective tools for encapsulating repetitive, model-specific testing logic into clean, readable commands. There is a more formal way of creating custom expectations with *testthat*, but for this purposes simple wrapper functions can be created. [16] The particular custom expectation functions are defined as follows.

```
#' Run model and check QALYs against an expected value
#'
#' @param data Baseline input data list.
#' @param expected_qalys The exact expected QALY value.
#' @param ... Arguments passed to `run_model()`.
#' @param tolerance Approximate equality.
#' @param label A label for the test, passed to `expect_equal()`.
check_model_qalys <- function(
                      data, expected_qalys, ...,
```

```
                      tolerance = NULL, label = NULL) {
  run <- run_model(data, ...)
  expect_equal(get_qalys(run), expected_qalys,
               tolerance = tolerance, label = label)
}

#' Run model and check Costs against an expected value
#'
#' @param data Baseline input data list.
#' @param expected_costs The exact expected Cost value.
#' @param ... Arguments passed to `run_model()`.
#' @param tolerance Approximate equality.
#' @param label A label for the test, passed to `expect_equal()`.
check_model_costs <- function(
                      data, expected_costs, ...,
                      tolerance = NULL, label = NULL) {
  run <- run_model(data, ...)
  expect_equal(get_costs(run), expected_costs,
               tolerance = tolerance, label = label)
}

#' Run model and check Life Expectancy against an expected value
#'
#' @param data Baseline input data list.
#' @param expected_le The exact expected LE value.
#' @param ... Arguments passed to `run_model()`.
#' @param label A label for the test, passed to `expect_equal()`.
check_model_le <- function(
                    data, expected_le, ..., label = NULL) {
  run <- run_model(data, ...)
  expect_equal(get_le(run), expected_le, label = label)
}

#' Compare model outputs from two different parameter sets
#'
#' @param extractor_fn Function to get the result (e.g., get_qalys).
#' @param comparison_fn testthat expectation function.
#' @param params_1 List of parameters for first `run_model()` call.
#' @param params_2 List of parameters for second `run_model()` call.
#' @param data Base dataset (passed to `run_model`).
#' @param label Label for test, passed to comparison function.
compare_model_runs <- function(extractor_fn, comparison_fn,
                               params_1, params_2, data, label = NULL) {

  run_1_args <- c(list(data = data), params_1)
  run_1 <- do.call(run_model, run_1_args)
  output_1 <- extractor_fn(run_1)

  run_2_args <- c(list(data = data), params_2)
  run_2 <- do.call(run_model, run_2_args)
  output_2 <- extractor_fn(run_2)
```

```
  comparison_fn(output_1, output_2, label = label)
}
```

We can now use these for the defined registry of health economic model tests as follows.

```
data(base_params)

test_that("QALY calculations", {

  # T04: Undiscounted QALYs (with u=1) should equal LE
  # This is a dynamic check
  # (comparing two *different* outputs from the *same* run),
  # so it doesn't fit the helper patterns
  # and is clearer to write manually.
  test_run_t04 <- run_model(base_params, discount_rate = 0,
                            u_healthy = 1, u_sick = 1)
  expect_equal(
    get_qalys(test_run_t04), get_le(test_run_t04),
    label = "T04: Undiscounted QALYs (with u=1) should equal LE")

  # T05: QALY gains exactly equal LYGs
  test_run_t05 <- run_model(base_params, u_healthy = 1, u_sick = 1)

  expect_equal(
    get_qalys(test_run_t05), get_le(test_run_t05),
    label = "T05: QALYs exactly equals LYGs when all utilities are 1")

  # T06: QALYs with standard discount < undiscounted QALYs
  compare_model_runs(
    extractor_fn = get_qalys,
    comparison_fn = expect_lt,
    params_1 = list(discount_rate = 0.035),
    params_2 = list(discount_rate = 0),
    data = base_params,
    label = "T06: Discounted should be less than undiscounted QALYs"
  )

  # T07: QALY gains after time 0 equal zero
  check_model_qalys(
    data = base_params,
    discount_rate = 1000,
    expected_qalys = c(without_drug = 0, with_drug = 0),
    tolerance = 0.01,
    label = "T07: High QALY discount rate reduces totals to 0"
  )
})

test_that("Cost calculations", {

  # Calculate baseline incremental QALYs
```

```r
base_res <- run_model(base_params)
base_qalys <- get_qalys(base_res)
inc_qalys <- unname(base_qalys["with_drug"] -
                    base_qalys["without_drug"])

# skip the rest of this block if assumption fails
testthat::skip_if_not(
  inc_qalys > 0,
  "Baseline incremental QALYs are <= 0.
   Skipping ICER directionality tests."
)

# T08: Set intervention costs = 0. ICER is reduced
# (Assuming baseline Incremental Effects are strictly positive)

compare_model_runs(
  extractor_fn = get_icer,
  comparison_fn = expect_lt,
  params_1 = list(c_intervention = 0),
  params_2 = list(), # Baseline run
  data = base_params,
  label = "T08: Intervention costs = 0 reduces ICER"
)

# T09: Increase intervention costs. ICER is increased
compare_model_runs(
  extractor_fn = get_icer,
  comparison_fn = expect_gt,
  params_1 = list(c_intervention = 10000),
  params_2 = list(),
  data = base_params,
  label = "T09: Increasing intervention costs increases ICER"
)

# T10: Costs with standard discount < undiscounted costs
compare_model_runs(
  extractor_fn = get_costs,
  comparison_fn = expect_lt,
  params_1 = list(discount_rate = 0.035),
  params_2 = list(discount_rate = 0),
  data = base_params,
  label = "T10: Discounted less than undiscounted costs"
)

# T11: Set cost discount rate to infinity. Costs equal zero t>0
check_model_costs(
  data = base_params,
  discount_rate = 1000,
  expected_costs = c(without_drug = 0, with_drug = 0),
  tolerance = 0.01,
  label = "T11: High cost discount rate reduces totals to 0"
```

```
  )
})

test_that("Population calculations using Life Expectancy (LE)", {

  # T01: Set relative treatment effects (odds ratio, hazard ratio,
  # risk ratio) = 1
  res_no_effect <- run_model(base_params, hr_treatment = 1)
  expect_equal(
    get_qalys(res_no_effect)["with_drug"],
    get_qalys(res_no_effect)["without_drug"],
    label = "T01: QALYs must be equal when relative treatment effect is 1"
  )

  expect_equal(
    get_le(res_no_effect)["with_drug"],
    get_le(res_no_effect)["without_drug"],
    label = "T01: LEs must be equal when relative treatment effect is 1"
  )

  # T02: If all transition probabilities to death are 0, LE is n_cycles
  n_cycles_test <- 50
  check_model_le(
    data = base_params,
    p_healthy_death = 0,
    p_sick_death = 0,
    n_cycles = n_cycles_test,
    expected_le = n_cycles_test,
    label = "T02: LE should be n_cycles when all p_death=0"
  )

  # T03: If all transition probabilities to death are 1, LE is 1 cycle
  check_model_le(
    data = base_params,
    p_healthy_death = 1,
    p_sick_death = 1,
    expected_le = 1,
    label = "T03: LE should be 1 when all p_death=1"
  )
})
```

A.2 Advanced Testing Approaches

A.2.1 Reproducible Stochastic Tests with *withr*

Probabilistic Sensitivity Analysis involves random sampling, so stochastic tests must set a seed to be reproducible. The *withr* package [29] provides `with_seed()`, which temporarily sets the RNG seed for the duration of a single expression and then restores the previous state. This avoids the side-effect of a `set.seed()` call persisting into subsequent tests and affecting their behaviour. Beyond checking simple statistical boundaries, these stochastic tests are vital for verifying clinical logic in a PSA. For instance, a test can ensure that

probabilistic sampling never accidentally generates a scenario where the utility of the 'Progressive' state is drawn higher than the utility of the 'Asymptomatic' state, which would violate basic clinical logic and silently skew the ICER.

```
test_that("T12: PSA beta samples remain within (0, 1)", {
  withr::with_seed(42, {
    psa_samples <- sample_psa_parameters(
      n_sim       = 1000,
      base_params = base_params
    )
    beta_cols <- grep("^utility_", names(psa_samples), value = TRUE)
    for (col in beta_cols) {
      expect_true(
        all(psa_samples[[col]] > 0 & psa_samples[[col]] < 1),
        label = paste0("T12: beta-sampled '", col,
                       "' must lie strictly in (0, 1)")
      )
    }
  })
})
```

B AI-Assisted Testing for Health Economic Models

The emergence of large language model (LLM) assistants such as Claude[1] represents a practical opportunity to accelerate and broaden the adoption of automated testing in health economic modelling. Rather than replacing the modeller's domain judgement, AI can reduce the authoring burden for predefined tests and boilerplate test code, and lower the entry barrier for analysts who are less comfortable with software engineering conventions.

B.1 Generating *testthat* Scaffolding

A major practical obstacle to writing tests is the time required to translate a conceptual check such as in Table 1 into syntactically correct `testthat` code. An LLM can help perform the translation almost instantaneously when given a clear description of the model's architecture and the intended checks.

An LLM will produce code structurally equivalent to the `test_that()` block presented in Section 6, complete with informative labels and appropriate `expect_lt()` assertions. The modeller's remaining task is to verify correctness, not to write from scratch. This task is analogous to a compiler warning: the tool catches the mechanical, while the expert retains responsibility for the semantically meaningful.

B.1.1 Standardised Context and Prompts

To be effective, these agents require a project-level context anchor, as a structured configuration file (e.g., `model-context.md`) that explicitly defines the project's persona, testing standards, and domain-specific vocabulary. By providing these metadata, modellers can

[1] `https://www.anthropic.com/claude`

guide agents to generate tests that align with the HTA protocols (e.g., ZIN guidelines) and the specific modular architecture of their model. The additional information facilitates better reproducibility and ensures that the agent does not simply suggest generic code.

The agreed health economic modelling tests can be included in the context as a source of truth. Table 1 is converted to enable a test *registry* approach. The `model-context.md` file acts as a bridge between the HTA Registry (the requirements) and the R implementation (the syntax). With the model context file in the project root, we can simply provide a prompt to the AI agent, such as: "Generate the testthat code for T06 and T12 based on the provided model-context.md." The agent will now automatically know which `run_model()` arguments to use, which getter functions are available, and how to label the test for auditability. An example `model-context.md` is provided in Section B.3. It is important to emphasise that the `model-context.md` workflow proposed here serves as an illustrative, practical example of how modellers might begin to safely structure AI-assisted testing, rather than an established or independently validated methodological standard. The application of LLMs within health economic modelling is still in its infancy. Consequently, while these prompt engineering frameworks offer immediate utility for drafting boilerplate code, future empirical research will be required to formally validate the safety, efficiency, and standardisation of AI-driven verification protocols across diverse modelling projects.

B.2 Limitations of AI-Assisted Testing

AI assistance introduces its own failure modes that modellers must remain vigilant about. First, LLMs can generate tests that are syntactically correct but semantically wrong, for example, asserting that discounted costs are *greater* than undiscounted costs, which inverts the intended logic. Second, over-reliance on AI-generated tests may create a false sense of completeness: an LLM cannot know which clinical assumptions are model-specific and which are universal. Third, as with any generated code, tests must be read and understood by the modeller before being committed; committing unreviewed tests undermines the very accountability they are intended to provide. In summary, AI is best thought of as an efficient first draft author, not as a subject-matter expert.

B.3 AI Agent Project Context

An example script for an AI agent markdown document is given below.

```
# AI Agent Project Context: Health Economic Model Verification

## 1. Persona & Scope
You are an expert Health Economist and R Software
Developer. Your goal is to write robust, transparent,
and reproducible unit tests for HTA models using the
‘testthat‘ package.

## 2. Structural Contract: Helper Functions
To ensure modularity and "separation of concerns," all
model interactions must follow these interface patterns:

* **Required Setters/Mutators** (Must return input object):
```

```
  * `set_discount_rate(input, discount_rate)`
  * `set_costs(input, scenario, values)`
  * `set_p_matrix(input, scenario, values)`
  * `set_utilities(input, scenario, values)`
  * `set_time_horizon(input, scenario, values)`

* **Required Getters/Accessors**:
  * `get_qalys(res)`: Extract total QALYs.
  * `get_costs(res)`: Extract total costs.
  * `get_le(res)`: Extract Life Expectancy.
  * `get_icer(res)`: Extract ICER.

* **Required Wrappers & Utilities**:
  * `run_model(data, ...)`: Mandatory wrapper.
  * `compare_model_runs(...)`: Encapsulate comparison
    logic for different model outputs.

## 3. Test Registry (Source of Truth)
All automated verification must align with these:

| ID | Group | Test Condition | Expected Result |
|----|-------|----------------|-----------------|
| T01 | Pop | Set relative effects = 1 | Equal LYGs & QALYs |
| T02 | Pop | Set mortality = 0 | LE = time horizon |
| T03 | Pop | Set mortality = 1 | LE = 1 |
| T04 | QALY | Set discount rate = 0 (u=1) | Total QALYs = LE |
| T05 | QALY | Set utilities = 1 | QALY gains = LYGS |
| T06 | QALY | Positive discount rate | Disc QALYs < Undisc |
| T07 | QALY | QALY discount rate = inf | QALYs t>0 -> 0 |
| T08 | Cost | Set intervention costs = 0 | ICER is reduced |
| T09 | Cost | Increase intervention costs | ICER is increased |
| T10 | Cost | Set cost discount rate = 0 | Disc costs = Undisc |
| T11 | Cost | Set cost discount rate = inf | Costs t>0 -> 0 |
| T12 | Input | Sampled parameters | Bounds (e.g. 0-1) |

## 4. Coding Standard & Implementation Rules
* **Modular Architecture**: "Separation of concerns" is
  mandatory. Do not write monolithic scripts.
* **TDD**: Define expectations based on requirements
  before writing model functions.
* **Naming**: Every `test_that()` block must start with
  its Registry ID.
* **Tolerance**: Use `tolerance = 0.01` for floating-
  point comparisons.
```

B.4 Mitigating AI Risks with Smoke and Regression Testing

As modellers increasingly utilise LLMs to draft model code or populate data structures, specific software testing paradigms become useful for catching AI-induced errors. *Smoke testing* is a preliminary level of testing designed to reveal simple, catastrophic failures. It

is effective for catching 'junk' data or severely hallucinated parameter structures generated by an LLM. A smoke test simply executes the model pipeline with the AI-generated inputs to verify it completes without fatal errors before the modeller invests time in deeper validation.

Furthermore, *regression testing* is useful when using an LLM to refactor or update an existing, working model. Because LLMs can occasionally hallucinate incorrect mathematical logic or inadvertently alter unrelated functions during an update, running a comprehensive suite of automated regression tests ensures that any new AI-generated modifications do not silently break the previously verified baseline logic of the model.